



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto *AnzenCore*

Curso: Calidad y Pruebas de Software

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco (2023076790)

Perez Peralta, Fabrizio Salvador Elias (2023077476)

Tacna – Perú

2026

AnzenCore
Informe de Factibilidad

Versión 2.0



CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	FSEPP - GFAA			27/03/2026	Versión Original
2.0	FSEPP - GFAA			04/07/2026	Corrección del Informe

ÍNDICE GENERAL

1.	Descripción del Proyecto	3
2.	Riesgos	3
3.	Análisis de la Situación actual	3
4.	Estudio de Factibilidad	3
4.1	Factibilidad Técnica	4
4.2	Factibilidad económica	4
4.3	Factibilidad Operativa	4
4.4	Factibilidad Legal	4
4.5	Factibilidad Social	5
4.6	Factibilidad Ambiental	5
5.	Análisis Financiero	5
6.	Conclusiones	5



Informe de Factibilidad

1. Descripción del Proyecto

1.1 Nombre del proyecto

AnzenCore – Analizador de Vulnerabilidades Moviles

1.2 Duración del proyecto

Se estima una duración de 3 meses, estructurada en 5 fases de desarrollo iterativo para garantizar un despliegue estable.

1.3 Descripción

El proyecto consiste en el desarrollo de una plataforma que automatiza la detección de vulnerabilidades en aplicaciones móviles Android (archivos APK) y de code smells en código fuente. A diferencia de un linter genérico, AnzenCore reimplementa un motor de análisis de APK (parsing del formato DEX, manifiesto y recursos) tanto en el backend Python como de forma 100% local/offline en una extensión de VS Code, generando hallazgos clasificados por severidad, CWE y categoría OWASP Mobile.

1.4 Objetivos

1.4.1 Objetivo general

Desarrollar una plataforma automatizada capaz de detectar vulnerabilidades de seguridad en aplicaciones móviles y code smells en código fuente, centralizando los hallazgos en un dashboard con trazabilidad histórica por usuario y escaneo.

1.4.2 Objetivos Específicos

- *Desarrollar un módulo de análisis de APK que detecte secretos hardcoded, HTTP inseguro, criptografía débil, WebView inseguro, IPs hardcodedas y librerías nativas, con evidencia y recomendación de remediación.*



- *Implementar un motor de análisis de código fuente (code smells y complejidad) que procese carpetas o archivos individuales y devuelva métricas por archivo (NOM, NOA, LOC, complejidad).*
- *Implementar una extensión de VS Code que ejecute el análisis de APK de forma 100% local y offline, y el análisis de code smells contra la API desplegada*
- *Diseñar y programar un dashboard interactivo (Streamlit) que visualice los escaneos, hallazgos y métricas de riesgo agrupadas por severidad.*
- *Desarrollar un servicio de exportación de reportes de hallazgos en formatos estructurados (PDF, CSV, XLSX, JSON)*
- *Implementar un sistema de autenticación basado en roles (admin/user) para garantizar que los hallazgos de seguridad solo sean accesibles por personal autorizado.*

2. Riesgos

Complejidad del Parsing: interpretar el formato binario DEX (bytecode Android) para extraer strings y tipos reales puede ser complejo si no se estandarizan las reglas de análisis.

Falsos positivos/negativos: posibilidad de que los patrones de detección (regex, co-ocurrencia de tipos) generen hallazgos incorrectos o dejen de detectar vulnerabilidades reales en apps ofuscadas.

Degradación de rendimiento: el análisis de APKs grandes o de proyectos con muchos archivos podría afectar el tiempo de respuesta de la API.

Riesgo	Descripción	Probabilidad	Impacto
--------	-------------	--------------	---------



Complejidad Parsing	Dificultad en traducir el bytecode DEX a hallazgos accionables.	Media	Alto
Falsos Positivos/Negativos	Detecciones incorrectas por patrones genéricos en apps ofuscadas.	Media	Alto
Rendimiento	Lentitud al analizar APKs o repositorios muy grandes.	Media	Medio
Disponibilidad	Ausencia temporal de un desarrollador clave.	Baja	Crítico

3. Análisis de la Situación actual

3.1 Planteamiento del problema

Los equipos que desarrollan aplicaciones móviles Android enfrentan una deficiencia crítica en la detección temprana de vulnerabilidades de seguridad y en el control de la calidad del código, provocada por la falta de un análisis



automatizado y centralizado, dividida en tres causas raíz que alimentan un ciclo de ineficiencia:

- *La ausencia de un análisis de seguridad estandarizado sobre los APK generados permite que vulnerabilidades móviles (OWASP Mobile) lleguen a producción sin ser detectadas.*
- *Los procesos de revisión de code smells y complejidad se realizan de forma manual o dispersa entre distintas herramientas, lo que consume tiempo del equipo y no permite medir objetivamente la deuda técnica acumulada del proyecto.*
- *No existe una correlación automatizada entre los hallazgos detectados y un historial centralizado por usuario y escaneo, lo que dificulta el seguimiento de remediaciones y las auditorías de seguridad a largo plazo.*

- **Árbol de problemas:**

Diagrama disponible en el repositorio del proyecto; ver también el diagrama entidad/relación de AnzenCore en el Diccionario de Datos.

El Problema Central

Vulnerabilidades de seguridad no detectadas y deuda técnica acumulada por la falta de análisis automatizado y centralizado en el ciclo de desarrollo de software.

Causas Raíz

El diagrama identifica seis áreas críticas donde los equipos de desarrollo fallan actualmente:

- ***Detección de vulnerabilidades deficiente: desconocimiento de hallazgos de seguridad hasta etapas tardías del desarrollo o incluso en producción.***



- **Falta de trazabilidad de hallazgos:** no hay historial centralizado de qué se analizó, cuándo y con qué resultado.
- **Herramientas dispersas y manuales:** uso de linters/escáneres desconectados entre sí y sin integración al editor, generando análisis inconsistentes.
- **Procesos de revisión manuales:** los desarrolladores revisan a mano patrones de vulnerabilidad conocidos, lo que es lento y propenso a errores.
- **Ausencia de evidencia estructurada:** los hallazgos, cuando se detectan, no incluyen evidencia, CWE u OWASP asociado, dificultando su priorización.
- **Falta de herramientas de análisis:** no existe un dashboard que permita ver severidad y volumen de hallazgos a lo largo del tiempo.

Efectos y Consecuencias

Estas fallas desencadenan una reacción en cadena que termina en:

- **Inmediatos:** vulnerabilidades móviles (secretos hardcodeados, HTTP inseguro, criptografía débil) llegan a producción sin ser detectadas.
- **A medio plazo:** incidentes de seguridad, retrabajo por vulnerabilidades encontradas tarde y acumulación de deuda técnica no medida.
- **Finales:** exposición a brechas de seguridad, pérdida de confianza de usuarios/clientes y mayor costo de remediación en producción frente a detectarlo en desarrollo.



3.2 Consideraciones de hardware y software

Se utilizará Python 3.12 (FastAPI + Streamlit) para la lógica del sistema y TypeScript para la extensión de VS Code. Para el despliegue en la nube se usará Terraform, aprovisionando la infraestructura en Azure Container Apps de forma reproducible y eliminando la configuración manual del servidor.

Categoría	Elemento	Descripción / Especificación
Hardware	Computadoras de desarrollo	Laptops con mínimo 8 GB RAM, procesador Intel Core i5 o superior, para codificación, pruebas locales y ejecución de contenedores Docker.
	Dispositivos para pruebas (VS Code / navegador)	Equipos usados para validar la extensión de VS Code y el dashboard responsive en distintos entornos.
	Red / Internet del equipo	Conectividad necesaria para el desarrollo colaborativo, despliegue

		en la nube y ejecución de la pipeline CI/CD.
Software	Python 3.12 (FastAPI)	Lenguaje y framework principal del backend (API). Procesa la lógica de negocio, el análisis de APK/código fuente y la conexión con la base de datos.
	Streamlit	Framework del dashboard interactivo: visualización de escaneos, hallazgos y métricas de severidad.
	TypeScript + esbuild (Extensión VS Code)	Extensión que ejecuta el análisis de APK 100% local/offline y consume la API para el análisis de código fuente.
	PostgreSQL (Supabase)	Sistema gestor de base de datos relacional gestionado, utilizado para usuarios, escaneos, hallazgos y artefactos.



		Garantiza integridad y concurrencia de datos.
	Docker	Plataforma de contenedores utilizada para empaquetar la API y el dashboard, garantizando que el entorno de desarrollo sea idéntico al de producción.
	Git y GitHub (Control de versiones + CI/CD)	Sistema de control de versiones distribuido y pipeline de GitHub Actions (tests, SonarQube, Snyk) para integración continua.
Infraestructura Cloud	Terraform	Infraestructura como código (IaC) utilizada para aprovisionar de forma reproducible el Azure Container Registry, el Container Apps Environment, el Log Analytics Workspace y los Container Apps de API y dashboard.



	Azure Container Apps (vía Terraform)	Servicio de hosting serverless con auto-escalado a cero para el despliegue de la API y el dashboard en producción.
--	---	--

4. Estudio de Factibilidad

4.1 Factibilidad Técnica

El proyecto es altamente viable. El stack (Python/FastAPI/Streamlit) es maduro y ya se encuentra desplegado en producción mediante Terraform sobre Azure Container Apps, lo que reduce el riesgo técnico de la infraestructura.

Categoría	Elemento	Descripción / Especificación
Hardware	Computadoras de desarrollo	Laptops con mínimo 8 GB RAM, procesador Intel Core i5 o superior, para codificación, pruebas locales y ejecución de contenedores Docker.
	Red / Internet del equipo	Conectividad necesaria para el desarrollo colaborativo, despliegue



		en la nube y ejecución de la pipeline CI/CD.
Software	Python 3.12 (FastAPI) + Streamlit	Backend de análisis y dashboard interactivo.
Infraestructura Cloud	Terraform + Azure Container Apps	Despliegue reproducible de la API y el dashboard, con escalado a cero.

4.2 Factibilidad Económica

4.2.1 Costos Generales

Comprende los equipos de cómputo de los integrantes del equipo y los insumos de escritorio necesarios para la documentación formal del proyecto.

Ítem	Descripción	Costo Estimado (S/)
<i>Equipos de cómputo</i>	<i>3 laptops de los integrantes del equipo (recursos propios). Cap. mínima: 8 GB RAM, i5, 256 GB SSD. No representa costo adicional al proyecto.</i>	<i>S/ 0.00 (recurso propio)</i>
<i>Insumos de escritorio</i>	<i>Materiales de papelería, folder, anillados y útiles para</i>	<i>S/ 80.00</i>



	<i>documentación formal del proyecto.</i>	
<i>GitHub Team (Control de versiones)</i>	<i>Plan GitHub Free: repositorios privados ilimitados para equipos pequeños, suficiente para el control de versiones del proyecto. \$0 USD.</i>	<i>S/ 0.00</i>
<i>Figma Free (Diseño UI/UX)</i>	<i>Plan Figma Free: diseño de interfaces y prototipado. Plan gratuito con hasta 3 proyectos activos, suficiente para el alcance del proyecto. \$0 USD.</i>	<i>S/ 0.00</i>
<i>Notion Free (Gestión de tareas)</i>	<i>Plan Notion Free: gestión de tareas, documentación técnica y backlog del proyecto. Plan gratuito ilimitado para equipos pequeños. \$0 USD.</i>	<i>S/ 0.00</i>
	SUBTOTAL COSTOS GENERALES	S/ 80.00

4.2.2 Costos operativos durante el desarrollo

Corresponde a los gastos recurrentes generados durante los tres meses de ejecución activa del proyecto: energía eléctrica, conectividad a internet y comunicaciones del equipo (proyecto de desarrollo 100% remoto, sin visitas a un taller físico).



Concepto	Descripción	Costo Mensual (S/)	Total 3 meses (S/)
<i>Energía Eléctrica</i>	<i>Consumo estimado de 3 laptops (8h/día, 6 días/semana) + iluminación y ventiladores del espacio de trabajo compartido.</i>	<i>S/ 75.00</i>	<i>S/ 225.00</i>
<i>Servicio de Internet</i>	<i>Plan de internet dedicado para desarrollo: acceso a repositorios, despliegue cloud (Terraform) y videollamadas de coordinación de equipo.</i>	<i>S/ 99.00</i>	<i>S/ 297.00</i>
<i>Comunicaciones del equipo</i>	<i>Llamadas y datos móviles adicionales para coordinación remota del equipo.</i>	<i>S/ 30.00</i>	<i>S/ 90.00</i>
	SUBTOTAL COSTOS OPERATIVOS	S/ 204.00	S/ 612.00

Post-lanzamiento, los costos operativos de desarrollo (energía, internet dedicado, comunicaciones del equipo) cesan; el único costo recurrente que continúa indefinidamente es la infraestructura cloud (Azure), detallada en la sección de Costos del Ambiente.

4.2.3 Costos del ambiente

Incluye la infraestructura provisionada mediante Terraform (Azure Container Registry, Container Apps Environment, Log Analytics y los dos Container Apps: api



y dashboard), el repositorio de código y la base de datos gestionada. El costo de cómputo de los Container Apps es variable: al escalar a cero réplicas cuando no hay tráfico, el costo real puede ser menor al aquí estimado (que asume un escenario de tráfico moderado).

Servicio	Detalle	Costo/mes (S/)	Total 3 meses (S/)
GitHub Team (Repositorios + CI/CD)	Plan GitHub Team: repositorios privados ilimitados, GitHub Actions para pipelines CI/CD, protección de ramas y revisión de código. \$4 USD/usuario/mes × 3 usuarios.	S/ 45.00	S/ 135.00
Azure Container Registry — Basic (Terraform: azurerm_container_registry.acr)	Registro privado de imágenes Docker (api y dashboard). SKU Basic, 10 GB de almacenamiento incluido. \$5 USD/mes.	S/ 18.75	S/ 56.25



Azure Log Analytics Workspace (Terraform: azurerm_log_analytics _workspace.log_analyt ics)	Monitoreo y logs de los Container Apps, retención de 30 días (SKU PerGB2018). Estimado para bajo volumen de ingestión. \$3 USD/mes.	S/ 11.25	S/ 33.75
Azure Container Apps — api + dashboard (Terraform: azurerm_container_ap p.api / .dashboard)	Cómputo serverless con escalado a cero: api (2 vCPU/4 GiB, 0-100 réplicas) y dashboard (1 vCPU/2 GiB, 0-10 réplicas). Estimado bajo tráfico moderado, neto del free grant mensual (180 000 vCPU-seg + 360 000 GiB- seg). \$11.88 USD/mes.	S/ 44.55	S/ 133.65
Azure Storage Account — backend remoto de Terraform (.tfstate)	Almacena el estado de Terraform de forma segura y compartida entre el equipo. Costo marginal por los pocos KB del archivo de estado. \$0.10 USD/mes.	S/ 0.40	S/ 1.20
	SUBTOTAL COSTOS DEL AMBIENTE	S/ 120.40	S/ 361.20

4.2.4 Costos de personal



Valoración de las horas-hombre dedicadas al diseño del motor de análisis de APK, desarrollo de la extensión de VS Code y redacción de pruebas unitarias exhaustivas.

Rol	Nombre	Horas/mes	Valor Hora (S/)	Total/mes (S/)	Total 3 meses (S/)
<i>Jefe de Proyecto / Dev. Principal</i>	<i>Gian Franco Arocutipa</i>	<i>96 h/mes</i>	<i>S/ 25.00</i>	<i>S/ 2,400.00</i>	<i>S/ 7,200.00</i>
<i>Analista Programador</i>	<i>Rodrigo Colque</i>	<i>80 h/mes</i>	<i>S/ 20.00</i>	<i>S/ 1,600.00</i>	<i>S/ 4,800.00</i>
<i>Analista Programador</i>	<i>Fabricio Ramos</i>	<i>80 h/mes</i>	<i>S/ 20.00</i>	<i>S/ 1,600.00</i>	<i>S/ 4,800.00</i>
			SUBTOTAL	S/ 5,600.00	S/ 16,800.00
			PERSONAL	0	0



4.2.5 Costos totales del desarrollo del sistema

Estimación final basada en el cronograma de ejecución del proyecto, incluyendo la infraestructura provisionada con Terraform.

N°	Concepto de Gasto	Monto (S/)	Acumulado (S/)
1	Personal (Jefe de Proyecto + 2 Analistas Programadores × 3 meses)	S/ 16,800.00	S/ 16,800.00
2	Hardware (disco externo de respaldo y accesorios menores)	S/ 150.00	S/ 16,950.00
3	Infraestructura y Servicios Cloud vía Terraform (3 meses)	S/ 361.20	S/ 17,311.20
4	Costos Operativos (Energía, Internet, Comunicaciones × 3 meses)	S/ 612.00	S/ 17,923.20
5	Costos Generales (Insumos + Herramientas de Software)	S/ 80.00	S/ 18,003.20
	TOTAL LÍNEA BASE		S/ 18,003.20
6	Reserva de Contingencia — 10% del total (riesgos técnicos identificados)	S/ 1,800.32	S/ 19,803.52



7	Reserva de Gestión — 5% del total (imprevistos administrativos)	S/ 900.16	S/ 20,703.68
	TOTAL PRESUPUESTO DEL PROYECTO		S/ 20,703.68

4.3 Factibilidad Operativa

El sistema es operativamente viable dado que se integra directamente en el flujo de trabajo de desarrollo existente (editor de código y pipeline CI/CD) sin requerir una reestructuración organizacional. Los perfiles de usuario (Administrador, Desarrollador) cuentan con interfaces diseñadas específicamente para sus necesidades:

- El Administrador dispondrá de un módulo de gestión de usuarios y reportes consolidados de todos los escaneos realizados por el equipo.*
- Los Desarrolladores accederán a un dashboard visual con los hallazgos por severidad y a la extensión de VS Code para analizar APKs sin salir del editor.*

4.4 Factibilidad Legal

El desarrollo se basa en librerías bajo licencias Open Source (FastAPI, Streamlit, fflate, esbuild), lo que permite su uso sin costos de regalías.

Además, el tratamiento de credenciales y hallazgos de seguridad se realiza bajo buenas prácticas (contraseñas cifradas con PBKDF2-HMAC-SHA256, secretos gestionados como variables sensibles en Terraform), respetando las normativas de protección de datos personales.



Categoría	Implementación en AnzenCore	Marco Legal
Datos Personales	Contraseñas cifradas (PBKDF2-HMAC-SHA256) y credenciales gestionadas como secretos en Terraform.	Ley N° 29733 (Perú)
Seguridad de Acceso	Control de acceso basado en roles (RBAC) y hashing de contraseñas.	Estándar ISO 27001
Licenciamiento	Software desarrollado bajo licencias Open Source (MIT / Apache).	Propiedad Intelectual
Confidencialidad	Gestión de secretos (SUPABASE_KEY, credenciales de registro) fuera del código fuente, vía variables sensibles de Terraform.	Protección de Procesos



El proyecto genera un impacto social positivo en múltiples dimensiones:

- *Fomenta una cultura de seguridad y calidad de código en el desarrollo de software, un área donde las vulnerabilidades suelen detectarse tarde. Los hallazgos con evidencia, CWE y OWASP Mobile empoderan a los desarrolladores con información objetiva y accionable.*
- *Contribuye al desarrollo de competencias en seguridad móvil (OWASP Mobile Top 10) y DevSecOps en los desarrolladores que usan la herramienta.*
- *El proyecto es desarrollado por estudiantes de la Universidad Privada de Tacna, lo que representa un aporte directo del ámbito académico a la comunidad de desarrollo de software.*
- *Reduce la brecha de conocimiento en seguridad móvil de equipos pequeños que no cuentan con un especialista dedicado en seguridad.*

El proyecto se alinea principalmente con el ODS 9 (Industria, Innovación e Infraestructura), al fomentar infraestructura tecnológica segura, y con el ODS 4 (Educación de Calidad), al ser desarrollado como parte de la formación académica de los estudiantes.

4.6 Factibilidad Ambiental

El proyecto tiene un impacto ambiental netamente positivo, alineado con políticas de modernización sostenible:

- *Es un proyecto enteramente digital: los reportes se exportan en PDF/CSV/XLSX bajo demanda, sin necesidad de impresión física.*
- *La automatización del análisis evita ejecutar herramientas de seguridad manuales y redundantes en múltiples máquinas, optimizando el uso de recursos de cómputo del equipo.*



- *El despliegue en Azure Container Apps con escalado a cero (min_replicas = 0, definido en Terraform) es energéticamente más eficiente que mantener servidores encendidos permanentemente, ya que solo consume recursos cuando hay tráfico real.*
- *La reducción del tiempo de detección de vulnerabilidades mediante análisis automatizado evita retrabajos y el consumo adicional de recursos computacionales en ciclos de corrección tardíos.*

5. Análisis Financiero

El análisis financiero del proyecto se enfoca en demostrar la viabilidad económica de la inversión mediante la evaluación de los flujos de caja generados por los ahorros operativos que AnzenCore producirá en un equipo de desarrollo de software. Se utilizan tres indicadores financieros estándar: la Relación Beneficio/Costo (B/C), el Valor Actual Neto (VAN) y la Tasa Interna de Retorno (TIR).

5.1 Justificación de la Inversión

5.1.1 Beneficios del Proyecto

Beneficios Tangibles:

- ***Reducción de costos por incidentes de seguridad: la detección temprana de vulnerabilidades móviles (OWASP Mobile) evita el costo de remediar hallazgos ya en producción, significativamente mayor que corregirlos en desarrollo.***
- ***Optimización del tiempo de revisión de código: al centralizar el análisis de code smells y vulnerabilidades en un dashboard, se reduce el tiempo que el equipo dedica a revisiones manuales dispersas.***



- **Aseguramiento de la trazabilidad de hallazgos: correlación automática entre cada escaneo y sus hallazgos (apk_scans → apk_findings), facilitando auditorías de seguridad y el seguimiento de remediaciones.**

Se proyecta una reducción cercana al 100% de vulnerabilidades móviles críticas que llegan a producción sin ser detectadas (para las categorías cubiertas por el motor de análisis) y una reducción estimada del 30%-40% en el tiempo dedicado a revisión manual de code smells.

Fuente del Beneficio	Impacto Operativo	Ahorro Mensual (S/)	Ahorro Semestral (S/)	Ahorro Anual (S/)
Detección Temprana de Vulnerabilidades	Evita retrabajo por vulnerabilidades detectadas en producción en vez de en desarrollo.	S/ 400.00	S/ 2,400.00	S/ 4,800.00
Automatización de Revisión de Code Smells	Reduce las horas dedicadas a la revisión manual de código.	S/ 250.00	S/ 1,500.00	S/ 3,000.00
Centralización y Trazabilidad de Hallazgos	Reduce el tiempo de auditoría y seguimiento manual de	S/ 150.00	S/ 900.00	S/ 1,800.00



	hallazgos.			
TOTAL	Ahorro total	S/ 800.00	S/ 4,800.00	S/ 9,600.00
BENEFICIOS	estimado			

Beneficios Intangibles:

- **Mayor confianza en la seguridad del software entregado:** los hallazgos con evidencia y clasificación CWE/OWASP dan al equipo información objetiva y accionable sobre el estado de seguridad real de sus aplicaciones.
- **Cultura de DevSecOps:** integrar el análisis en el editor (extensión de VS Code) y en el pipeline (GitHub Actions) fomenta la detección temprana como práctica habitual, no como un paso adicional.
- **Reducción de la carga cognitiva del equipo:** al centralizar hallazgos de múltiples fuentes (APK, código fuente, URL, GitHub) en un solo dashboard, se reduce la fricción de usar herramientas dispersas.

Beneficio Intangible	Descripción del Impacto
Confianza en la seguridad del software	Hallazgos con evidencia, CWE y OWASP Mobile dan información objetiva y accionable sobre el riesgo real.
Cultura DevSecOps	Integración del análisis en el editor (VS Code) y el pipeline CI/CD fomenta la detección temprana como práctica habitual.

<i>Reducción de fricción entre herramientas</i>	<i>Centraliza hallazgos de APK, código fuente, URL y GitHub en un solo dashboard.</i>
<i>Sostenibilidad operativa</i>	<i>El escalado a cero en Azure Container Apps (definido en Terraform) reduce el consumo de recursos cuando no hay tráfico.</i>
<i>Escalabilidad y oportunidad SaaS</i>	<i>La arquitectura (API + dashboard + extensión) permite ofrecer AnzenCore como servicio a otros equipos de desarrollo en el futuro.</i>

5.1.2 Criterios de Inversión

Indicador	Valor Calculado	Interpretación
Inversión Inicial (C)	S/ 18,003.20	Costo total real del proyecto (personal, hardware, infraestructura cloud vía Terraform, operativos y generales).
Beneficio Neto Anual	S/ 8,155.20	Ahorro operativo anual proyectado (S/ 9,600 bruto) neto del costo recurrente de infraestructura cloud (S/ 1,444.80/año).
Relación B/C	B/C = 1.09	Por cada sol invertido, el proyecto recupera S/ 1.09 en valor presente de beneficios (horizonte 3 años,



		COK 12%). $B/C > 1$: proyecto económicamente viable.
VAN (COK = 12% anual)	S/ 1,585.00	VAN > 0. El proyecto genera valor positivo real en un horizonte de 3 años.
TIR	TIR $\approx$ 17%	TIR (17%) > COK (12%). El proyecto supera el costo de oportunidad del capital.

5.1.2.1 Relación Beneficio/Costo (B/C)

Costos (C): Se estima una inversión de S/ 18,003.20 (línea base), que cubre personal de desarrollo, hardware, infraestructura cloud provisionada con Terraform y costos operativos durante los 3 meses de ejecución.

Beneficios (B): El ahorro anual bruto estimado por la detección temprana de vulnerabilidades y la reducción del tiempo de revisión manual asciende a S/ 9,600.00. Descontando el costo recurrente de infraestructura cloud (S/ 1,444.80/año), el beneficio neto anual es de S/ 8,155.20.

$$B/C = 1.09$$

5.1.2.2 Valor Actual Neto (VAN)



Este indicador mide el valor actual de los ahorros netos que AnzenCore generará durante un horizonte de evaluación de 3 años.

Inversión Inicial: S/ 18,003.20.

Ahorro Neto Mensual Proyectado: S/ 679.60 a partir del mes 4 (lanzamiento del sistema), ya descontado el costo recurrente de infraestructura cloud.

Tasa de Descuento (COK): 12% anual.

Tras el cálculo (horizonte de 3 años, flujos anuales descontados al 12%), el VAN es de S/ 1,585.00. Al ser mayor a cero, se confirma que el proyecto es financieramente rentable y aporta un valor positivo al equipo de desarrollo.

5.1.2.3 Tasa Interna de Retorno (TIR)

Representa la rentabilidad anual de los recursos destinados al desarrollo del motor de análisis de vulnerabilidades y sus algoritmos de detección.

TIR Estimada: 17%

Costo de Oportunidad (COK): 12%.

Con una TIR de aproximadamente 17%, superior al costo de oportunidad del capital (12%), se acepta el proyecto. Esto indica que la inversión en AnzenCore es más rentable que el costo de oportunidad del capital, aunque con un margen más moderado que en un modelo de negocio con múltiples clientes.



5.1.3 Flujo de Caja

El flujo de caja proyecta los movimientos de dinero del proyecto desde la fase de desarrollo (Meses 0-3) hasta el cierre del horizonte de evaluación de 3 años (36 meses). Las cifras están en soles (S/).

Supuestos del modelo:

- Mes 0 (Pre-desarrollo): adquisición de hardware menor y herramientas antes de iniciar.
- Meses 1-3 (Desarrollo): pago de personal, infraestructura cloud activa (Terraform) para staging y costos operativos completos.
- Mes 4 (Lanzamiento): fin del desarrollo. AnzenCore queda desplegado en producción y se materializan los ahorros operativos por detección temprana de vulnerabilidades.
- Meses 4-36: el costo de infraestructura cloud (Azure Container Apps, ACR, Log Analytics) es recurrente y permanente, parcialmente compensado por el escalado a cero en periodos sin tráfico.
- El flujo acumulado alcanza su punto de recuperación (payback) proyectado alrededor del mes 27.
- Los costos de infraestructura cloud (provisionada con Terraform) son recurrentes y permanentes desde el Mes 4 (una vez en producción).
- Los costos operativos de desarrollo (personal, energía, internet dedicado) se reducen al finalizar la Fase 3 (Mes 3).

En base a 3 años



Período	Total Ingresos (S/)	Total Egresos (S/)	Flujo Neto (S/)	Flujo Acumulado (S/)
AÑO 0 (Pre-dev + Desarrollo, meses 0-3)	-	18,003.20	(18,003.20)	(18,003.20)
AÑO 1 (Meses 4-15)	9,600.00	1,444.80	8,155.20	(9,848.00)
AÑO 2 (Meses 16-27)	9,600.00	1,444.80	8,155.20	(1,692.80)
AÑO 3 (Meses 28-36)	7,200.00	1,083.60	6,116.40	4,423.60

Concepto	Valor (S/)
Inversión Total (Línea Base)	18,003.20
Reserva de Contingencia (10%)	1,800.32
Reserva de Gestión (5%)	900.16
TOTAL PRESUPUESTO DEL PROYECTO	20,703.68
Ahorro Bruto Mensual Proyectado (Mes 4+)	800.00
Costo Infraestructura Cloud/mes (Terraform)	120.40
Ahorro Neto Mensual Proyectado (Mes 4+)	679.60
Tasa de Descuento — COK (anual)	12.00%



Indicador	Valor	Criterio de Decisión
Inversión Inicial (C)	18,003.20	Costo total real (Personal + Hardware + Cloud vía Terraform + Operativos + Generales)
Beneficio Neto Anual	8,155.20	Ahorro bruto proyectado: S/ 800/mes × 12 meses, neto del costo de infraestructura cloud
Relación Beneficio/Costo (B/C)	1.09	$B/C > 1 \rightarrow$ proyecto económicamente viable (horizonte 3 años, COK 12%)
VAN (COK = 12% anual)	1,585.00	$VAN > 0 \rightarrow$ el proyecto genera valor positivo real
TIR	$\approx 17\%$	$TIR (17\%) > COK (12\%) \rightarrow$ se acepta el proyecto
Payback (recuperación)	$\approx$ Mes 27	El flujo acumulado alcanza cero aproximadamente en el mes 27

6. Conclusiones

El estudio de factibilidad confirma que el proyecto de implementación de AnzenCore es viable en todas sus dimensiones evaluadas:

•Técnicamente, el stack tecnológico seleccionado (Python 3.12, FastAPI, Streamlit, PostgreSQL/Supabase, Docker, Terraform) es maduro y ya se encuentra desplegado en producción, asegurando una implementación estable. El uso de



Terraform como infraestructura como código elimina la configuración manual del servidor, reduciendo significativamente el riesgo operativo del despliegue.

•Económicamente, la inversión total asciende a S/ 20,703.68 (línea base S/ 18,003.20 más reservas del 15%). La infraestructura cloud, provisionada y documentada mediante Terraform (Azure Container Registry, Container Apps Environment, Log Analytics y los Container Apps de API y dashboard), representa un costo recurrente estimado de S/ 120.40/mes bajo tráfico moderado, parcialmente mitigado por el escalado a cero. Bajo esta premisa, el proyecto alcanza un Valor Actual Neto (VAN) de S/ 1,585.00, una Tasa Interna de Retorno (TIR) de aproximadamente 17% y una relación Beneficio/Costo (B/C) de 1.09 en un horizonte de 3 años, consolidándose como una inversión financieramente viable.

•Operativamente, el sistema se integra de forma natural en el flujo de trabajo de desarrollo mediante interfaces diseñadas por rol (Administrador, Desarrollador) y una extensión de VS Code, lo que optimiza la detección de vulnerabilidades sin interrumpir las actividades diarias del equipo.

•Legalmente, el desarrollo cumple estrictamente con la Ley N° 29733 de Protección de Datos Personales del Perú. La implementación de controles de acceso basados en roles y la gestión de secretos mediante Terraform garantizan la seguridad y confidencialidad de la información almacenada.

•Social y Ambientalmente, el proyecto promueve una cultura de seguridad y calidad de código, y su despliegue con escalado a cero en Azure Container Apps reduce el consumo de recursos frente a servidores permanentemente encendidos. Además, eleva el estándar de formación en seguridad móvil de los estudiantes de la Universidad Privada de Tacna.

La transición de procesos manuales de revisión de seguridad a una plataforma automatizada con trazabilidad por escaneo garantiza un control estricto de los



hallazgos y elimina los errores humanos en su seguimiento. La arquitectura del sistema (API, dashboard y extensión de VS Code), desplegada de forma reproducible con Terraform, permite su evolución hacia un modelo escalable de servicio para más equipos de desarrollo en el futuro.